

Executive Architecture Overview

Live EPL Predictive Engine Pipeline

System Pipeline Overview

When a live EPL fixture kicks off, the engine operates as a continuous 4-stage pipeline running in memory and across GPU registers:

```
[Live Opta WebSocket Stream]
  -> Stage 1: RAM Ledger Accumulation (n_live, T_live)
      |
[15-Season Historical DB]
  -> Stage 2: GPU K-NN Tactical Indexing (Q_KNN via torch.cdist)
      |
[Split EPL Priors Q_home/away]
  -> Stage 3: Tri-Modal Bayesian Decay Blending (Q_active(t))
      |
      Stage 4: Zero-Branching PyTorch GPU Monte Carlo Sim
      |
[Live Win/Draw/Away Probs & xG] <---> [De-Vigged Bookie Odds]
```

1 Phase 1: Data Infrastructure & Asymmetric Priors (Offline Foundation)

Before touching live streams, we must engineer the static mathematical foundation using your 15 seasons (2011/12 to 2025/26) of Opta event data.

1. 15-Season Data Standardization & Entity Mapping

- **Club Slug Registry:** Build an automated mapping layer (CLUB_ID_MAP) to standardize club naming evolutions across 15 years (e.g., handling relegations, promotions, and naming conventions like *Brighton & Hove Albion* vs. *Brighton*).
- **Dynamic Strength Scaling:** Replace international Elo ratings with a domestic club strength metric (such as Club Elo, Opta Power Rankings, or SPI) to accurately capture the wide variance between title contenders and newly promoted sides.

2. Dual Asymmetric Global Priors (Q_{home} and Q_{away})

- **Retire Tournament Sterilization:** Permanently delete `neutralise_global_prior()` and the static `HostAdvantageModifier`. In a domestic league, home-field advantage is a structural spatial reality, not a flat percentage boost.
- **Split Venue Generator Matrices:** Slice the ~5,700 historical matches by acting team venue to generate two distinct 12-state transition baseline matrices:

$$\lambda_{ij}^{Q_{\text{home}}} = \frac{\sum n_{ij}^{\text{all home clubs}}}{\sum T_i^{\text{all home clubs}}}, \quad \lambda_{ij}^{Q_{\text{away}}} = \frac{\sum n_{ij}^{\text{all away clubs}}}{\sum T_i^{\text{all away clubs}}}$$

- **Asymmetric Conjugate Updating:** Update each specific club's historical home/away ledger against its respective venue prior via Dirichlet-Multinomial conjugate updating before match kickoff.

2 Phase 2: Hyperparameter & Tactical Non-Stationarity Optimization

With 15 seasons of data, treating a 2011/12 match (4-4-2, direct crossing, lower pressing tempo) identically to a 2025/26 match (high-line positional pressing, build-up from the goalkeeper) will degrade prediction accuracy. We must mathematically model tactical evolution.

1. Exponential Season Time-Decay (γ)

- Implement a historical time-decay weighting function across seasons when compiling baseline transition intensities and future K-NN ledgers.
- Assign each historical match a weight based on its age in seasons (Δs):

$$w_{\text{season}} = e^{-\gamma \cdot \Delta s}$$

- **Why:** This ensures older seasons contribute to overall sample stability and prevent zero-count division errors, but modern pressing-era trajectories dominate the quantitative baseline.

2. The Offline Optimization Manifold (α, β, γ)

Before running live, build an optimization loop to minimize pre-match Market RMSE against historical closing bookmaker odds across the last 5 seasons:

- α (**Prior Weight**): Calibrate the Dirichlet-Multinomial conjugate prior weight governing how aggressively a club's recent form overrides the global league prior.
- β (**Elo Hazard Scaling**): Tune the exponential scaling factor ($\exp(\beta \cdot \Delta \text{Elo})$) that accelerates or decelerates on-ball progression and scoring intensities based on rating differentials.
- γ (**Season Decay**): Optimize the historical decay constant to find the exact mathematical balance between sample volume and tactical relevance.

3 Phase 3: K-NN Tactical Indexing & Vector Database Compilation

To interpret live tactical dominance before match event counts statistically mature (Minutes 0 to 20), the engine queries a pre-compiled vector database of historical fixtures.

1. Recalibrating EPL 5D Tactical Norms (μ and σ)

- In `TacticalVectoriser`, compute the global mean vector (μ) and standard deviation vector (σ) across all 15 EPL seasons for the 5D feature space:

$$\vec{V}_{\text{live}}(t) = \left[\text{Field Tilt}, \quad P_{\text{prog,H}}, \quad P_{\text{prog,A}}, \quad \text{Tempo} \left(\sum \lambda_{\text{live}} \right), \quad g_H - g_A \right]$$

- Expect domestic Markovian tempo (V_3) and forward progression ratios (V_1, V_2) to establish significantly higher baselines than international tournament norms.

2. Offline Minute-Slice Compilation (.pt Files)

- Run `compile_historical_database` across all $\sim 5,700$ matches, slicing them into 60-second intervals from Minute 1 to Minute 90.
- Export production-ready `.pt` tensor files per minute containing:
 - Normalized historical 5D feature matrices (shape: $M \times 5$).
 - Future transition count matrices (n_{future} , shape: $M \times 12 \times 12$), pre-weighted by our season decay factor w_{season} .
 - Future holding time vectors (T_{future} , shape: $M \times 12$), pre-weighted by w_{season} .

4 Phase 4: Live WebSocket Ingestion & Continuous State Tracking

When the live match begins, we must transform incoming event packets into continuous-time mathematical structures without triggering OS memory garbage collection.

1. Asynchronous WebSocket Reader (LiveEventScraper)

- Build an async streaming listener that ingests Opta live event JSON payloads in real time.
- Maintain three continuous trackers in CPU RAM:
 1. **Global Match Clock (t):** Elapsed match time in seconds.
 2. **Scoreboard Tensor:** Current goals scored $[g_H, g_A]$ stored in pinned RAM.
 3. **Active Ball State Index:** Integer from 0 to 11 mapping spatial zone and possession.

2. Pinned RAM Ledger Accumulation

- For every incoming touch event, apply the 15-second dead-time stoppage cap (`apply_stoppage_cap`) to preserve tactical realism.
- Incrementally update the running 12x12 transition count matrix (n_{ij}^{live}) and 12-element holding time vector (T_i^{live}).
- Maintain all ledgers in **pinned CPU memory** (`pin_memory=True`) to allow non-blocking Direct Memory Access (DMA) transfers to the GPU.

5 Phase 5: Tri-Modal Bayesian Exponential Decay Engine

At any timestamp t , the active generator matrix $\lambda_{ij}^{\text{active}}(t)$ fed into the GPU simulator is a linear combination of our three predictive pillars.

Match Phase	Dominant Component	Tactical & Mathematical Justification
0 to 20 Mins	Pre-Match Prior (~70%)	Live sample size T_{live} is too small; protects the engine from overreacting to early random variance or kickoff flukes.
20 to 65 Mins	K-NN Neighbors (~55%)	Pitch reality is established. Queries the database for the 50 most identical historical tactical battles to map future trajectory.
65 to 90 Mins	Raw Live Pitch (~60%)	Historical priors decay to near-zero. The model converges strictly to active player fatigue, desperate end-game tactics, and scoreboard reality.

1. Continuous Decay Kinetics

Calculate exponential weights governed by a 45-minute half-life ($T_{1/2} = 2700\text{s}$, $\lambda_{\text{decay}} = \ln(2)/2700$):

$$w_{\text{pre}}(t) = e^{-\lambda_{\text{decay}} \cdot t}$$

$$w_{\text{live}}(t) = (1 - w_{\text{pre}}(t)) \cdot \left(\frac{t}{5400}\right)^{1.5}$$

$$w_{\text{KNN}}(t) = 1.0 - w_{\text{pre}}(t) - w_{\text{live}}(t)$$

2. Zero-Branching GPU Neighbor Search (`TacticalKNNIndexer`)

- Every 60 seconds (or upon a significant event), vectorize the live match into $\vec{V}_{\text{live}}(t)$ and push it to the GPU.
- Execute native PyTorch Euclidean distance calculations via `torch.cdist()` against the active minute-bucket `.pt` database:

$$d(\vec{V}_{\text{live}}, \vec{V}_m) = \sqrt{\sum_{k=1}^5 (V_{\text{live},k} - V_{m,k})^2}$$

- Extract the $K = 50$ nearest historical neighbors and aggregate their time-decayed future ledgers to generate the dense pseudo-prior $\lambda_{ij}^{\text{KNN}}(t)$.

3. Row-Wise Matrix Blending & CTMC Validation

- Fuse the three matrices element-wise in GPU VRAM:

$$\lambda_{ij}^{\text{active}}(t) = w_{\text{pre, eff}} \cdot \lambda_{ij}^{\text{pre}} + w_{\text{KNN, eff}} \cdot \lambda_{ij}^{\text{KNN}} + w_{\text{live, eff}} \cdot \left(\frac{n_{ij}^{\text{live}}}{T_i^{\text{live}} + \epsilon}\right)$$

- Apply row-wise dynamic reallocation: if a state has not been visited live ($T_i^{\text{live}} \leq \epsilon$), automatically reallocate w_{live} proportionally back to the prior and K-NN weights to prevent artificial exit-rate deflation.
- Enforce CTMC validity by zeroing the diagonal and setting $\lambda_{ii}^{\text{active}} = -\sum_{j \neq i} \lambda_{ij}^{\text{active}}$.

6 Phase 6: Short-Horizon PyTorch Monte Carlo Simulation

With the valid infinitesimal generator matrix $\lambda_{ij}^{\text{active}}(t)$ residing in GPU registers, we launch the zero-branching simulation engine.

1. Dynamic Parameter Injection

- Modify `run_pytorch_monte_carlo_tensor` to simulate over the shortening remaining time delta:

$$\Delta t = 5400.0 - t_{\text{current}}$$

- Initialize parallel GPU tensors for `home_goals` and `away_goals` using the active live scoreboard $[g_H, g_A]$ instead of zeros.
- Inject the active ball state index across all parallel simulation paths:

```
current_states = torch.full((num_sims,), active_ball_idx, device=device,
                             dtype=torch.long)
```

2. Latency Budget & Parallel Execution

- Launch 3,000 to 5,000 parallel paths on the RTX 4070 Ti.
- Because the PyTorch simulator operates with **0% branching** (no conditional `if` statements or CPU synchronization inside execution bursts), completing short-horizon paths ($\Delta t < 5400\text{s}$) will take approximately **10 to 25 milliseconds**.
- **Strict Latency Budget Audit:** Ensure total pipeline execution remains comfortably below the 200ms threshold:

$$\text{Latency}_{\text{total}} = t_{\text{Ingest}}(\sim 5\text{ms}) + t_{\text{KNN}}(\sim 40\text{ms}) + t_{\text{Decay}}(\sim 10\text{ms}) + t_{\text{Sim}}(\sim 20\text{ms}) \leq \mathbf{75\text{ms}}$$

7 Phase 7: Quantitative Benchmarking & Market Alpha Detection

The final phase proves whether our continuous-time mathematical architecture generates actionable edge against commercial betting firms.

1. Historical In-Play Replay Harness

- Build a backtesting emulator that reads archived 15-season Opta JSON files and pushes event packets through the WebSocket ingestion engine at 10x or 100x speed.
- At every 60-second timestamp, record the model's output probabilities $[P(\text{Home}), P(\text{Draw}), P(\text{Away})]$ and expected goals (xG).

2. De-Vigged Market RMSE Auditing

- Ingest historical, timestamped in-play bookmaker odds (e.g., Bet365 or Pinnacle live odds recorded at 1-minute intervals).
- Remove bookmaker overround (vig) using margin-proportional or power de-vigging methods to uncover true market implied probabilities.
- Calculate real-time Market RMSE across all 90 minutes:

$$\text{RMSE}_{\text{market}}(t) = \sqrt{\frac{1}{3} \sum_{k \in \{H,D,A\}} (P_{\text{model},k}(t) - P_{\text{market},k}(t))^2}$$

3. Alpha Detection & Edge Quantification

- **Latency Exploitation:** Commercial bookmakers often suspend betting markets or rely on manual traders/simple Poisson adjustments following a major tactical shift, red card, or sudden surge in Zone 4 Field Tilt.
- **The Window of Opportunity:** Audit backtest logs specifically during **Minutes 20 to 65**. This is where our K-NN Tactical Pseudo-Prior (Q_{KNN}) identifies structural momentum shifts before raw scoreboard metrics or standard bookmaker algorithms adjust, generating positive Expected Value (EV) betting opportunities.